# STATION-LEVEL EV CHARGING DEMAND FORECASTING

**A PROJECT BASED LEARNING REPORT**

***Submitted by***

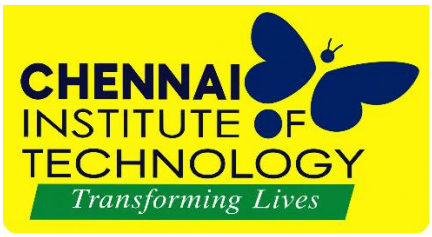
**M PRAVEEN VIGNESH (210425205091)**
**J FARHAN SOHAIL (210425205059)**

***Submitted in partial fulfilment of the requirements***
***for the***
***Project Based Learning component of Machine Learning***

**BACHELOR OF TECHNOLOGY**

***in***

**INFORMATION TECHNOLOGY**




**CHENNAI INSTITUTE OF TECHNOLOGY, CHENNAI**




**Affiliated to Anna University, Chennai**
**(Autonomous)**

OCTOBER 2026

## Vision of the Institute:

To be an eminent centre for Academia, Industry and Research by imparting knowledge, relevant practices and inculcating human values to address global challenges through novelty and sustainability.

## Mission of the Institute:

**IM1.** To create next generation leaders by effective teaching learning methodologies and instill scientific spark in them to meet the global challenges.

**IM2.** To transform lives through deployment of emerging technology, novelty and sustainability.

**IM3.** To inculcate human values and ethical principles to cater the societal needs.

**IM4.** To contribute towards the research ecosystem by providing a suitable, conducive and competent environment.

SIRAGU

# DEPARTMENT OF INFORMATION TECHNOLOGY

## Vision of the Department:

To lead in the evolving fields of Information Technology by fostering industry-academia collaboration, advancing research, and promoting sustainable practices, preparing students to develop innovative, technology-driven solutions for real-world challenges.

## Mission of the Department:

**DM1.** To nurture future leaders by adopting innovative teaching methodologies, inspiring a passion for emerging technologies, and equipping students to tackle global challenges effectively.

**DM2.** To empower students to develop sustainable solutions that enhance quality of life, transforming communities through innovation and technology.

**DM3.** To instill human values, ethical principles, and professionalism in students, preparing them to contribute meaningfully to society and uphold ethical standards in the industry.

**DM4.** To strengthen the research ecosystem by promoting collaboration among academia, industry, and R&D establishments, enabling impactful research and technological progress.

**DM5.** To inspire entrepreneurship by fostering creativity, leadership, and problem-solving skills, enabling students to develop impactful solutions and successful start-ups.

SIRAGU

# BONAFIDE CERTIFICATE

This is to certify that the Project–Based Learning report titled **"STATION-LEVEL EV CHARGING DEMAND FORECASTING"** is a Bonafide record of work carried out by **M PRAVEEN VIGNESH (210425205091)** and **J FARHAN SOHAIL (210425205059)** of the Department of Information Technology, Chennai Institute of Technology, as part of the continuous, mentor–guided Project-Based Learning (PBL) component of the Machine Learning course (IT5305) during the academic year 2026–2027 under my supervision.

**SIGNATURE**
Dr. A. R. Kavitha, M.E., Ph.D.,
**Professor and Head,**
Dept. of Information Technology,
Chennai Institute of Technology,
Chennai – 69.

**SIGNATURE**
Dr. D. Nithya Baskaran, M.E., Ph.D.,
**Associate Professor,**
Dept. of Information Technology,
Chennai Institute of Technology,
Chennai – 69.

Submitted for the final review held on ……

**Internal Examiner** **External Examiner**

# DECLARATION

We jointly declare that the PBL report on **"STATION-LEVEL EV CHARGING DEMAND FORECASTING"** is the result of original work done by us and, to the best of our knowledge, similar work has not been submitted to **"ANNA UNIVERSITY, CHENNAI"** for the requirement of the Degree of **BACHELOR OF TECHNOLOGY**. This PBL report is submitted in partial fulfilment of the requirement for the award of the Degree of Information Technology.

SIRAGU

Signature

M PRAVEEN VIGNESH (210425205091)
J FARHAN SOHAIL (210425205059)

Place: Chennai
Date:

# ACKNOWLEDGEMENT

We wish to express our sincere gratitude to our honorable Chairman, **SHRI. P. SRIRAM** for providing immense facilities at our institution.

We are very proudly rendering our thanks to our Principal, **Dr. A. RAMESH, M.E., Ph.D.,** for the facilities and the encouragement given by him for the progress and completion of our project.

We would like to express special thanks of gratitude to our Dean, **Dr. V. SRINIVASA RAO, M.E., Ph.D.,** who has been the key spring of motivation to us throughout the completion of our course and project work.

We proudly render our immense gratitude to the Head of the Department, **Dr. A. R. KAVITHA, M.E., Ph.D.,** for her effective leadership, encouragement, and guidance in the project.

We would like to extend our thanks to the Project Coordinator **Dr. D. Nithya Baskaran, M.E., Ph.D., Associate Professor,** Department of Information Technology, for their valuable suggestions throughout this project.

We wish to acknowledge the help received from the class advisors **Mr. S. Kumaragurubaran, M.C.A., M.E.,** of the Department of Information Technology, and others for providing valuable suggestions and for the successful completion of the project.

M PRAVEEN VIGNESH (210425205091)

J FARHAN SOHAIL (210425205059)

# ABSTRACT

Station-Level EV Charging Demand Forecasting is a machine learning project that forecasts next-hour electric vehicle (EV) charging demand, in kWh per hour, for a workplace charging site. It uses 6,490 real charging sessions from 53 stations in the Caltech ACN-Data dataset, downloaded through its public REST API by a retry-safe script that saves every page as it arrives. An average station has no demand in 79% of hours, so sessions are aggregated to whole-site hourly demand, giving 2,555 hourly rows between April and August 2018. Six backward-looking features (hour of day, weekday, and demand 1 h, 2 h, 24 h and 168 h earlier) are engineered and the data are split chronologically. Against a same-hour-last-week baseline (MAE 8.86 kWh/h), Linear Regression reaches 4.92 and a cross-validated Random Forest 4.41 kWh/h (50.2% better) on 478 held-out hours; an ablation shows most of this gain comes from the 1 h and 2 h lags, and without them the gain falls to 13.9%. A leakage test confirms the features look only backward. Results are served by a local web dashboard. The system is a decision-support prototype that under-predicts sharp peaks.



**Keywords:** EV charging demand, time-series forecasting, Random Forest, lag features, ACN-Data

# TABLE OF CONTENTS


| CHAPTER NO | TITLE | PAGE NO |
|---|---|---|
| 1 | **INTRODUCTION** | 1 |
| | 1.1 Background | 1 |
| | 1.2 Driving Question | 1 |
| | 1.3 Objectives | 2 |
| | 1.4 Scope and Limitations | 2 |
| 2 | **LITERATURE SURVEY** | 3 |
| | 2.1 Related Approaches | 3 |
| | 2.2 Summary Table | 3 |
| 3 | **PROJECT PLANNING AND TEAM ORGANISATION** | 5 |
| | 3.1 Weekly PBL Progress Log | 5 |
| | 3.2 Requirements | 5 |
| | 3.3 Feasibility | 6 |
| 4 | **ITERATIVE DESIGN AND DEVELOPMENT** | 7 |
| | 4.1 System Architecture | 7 |
| | 4.2 Iteration 1 — Baseline | 7 |
| | 4.3 Iteration 2 — Refinement | 8 |
| | 4.4 Final Approach | 8 |
| | 4.5 Training Procedure | 9 |
| 5 | **IMPLEMENTATION** | 10 |
| | 5.1 Module Description | 10 |
| | 5.2 Key Code Snippets | 11 |
| | 5.3 User Interface / Demo | 12 |
| 6 | **RESULTS AND DISCUSSION** | 15 |
| | 6.1 Evaluation Metrics | 15 |
| | 6.2 Results Across Iterations | 15 |
| | 6.3 Discussion | 16 |
| | 6.4 Limitations | 17 |
| 7 | **TEAM REFLECTION AND LEARNING OUTCOMES** | 19 |
| | 7.1 Individual Reflections | 19 |
| | 7.2 Team Learning | 19 |
| | 7.3 Course Outcomes — Evidence Summary | 20 |

| CHAPTER NO | TITLE | PAGE NO |
|---|---|---|
| 8 | **CONCLUSION AND FUTURE SCOPE** | 21 |
| | 8.1 Conclusion | 21 |
| | 8.2 Future Scope | 21 |
| | **REFERENCES** | 22 |
| | **APPENDIX** | 23 |

# LIST OF FIGURES


| FIGURE NO | TITLE | PAGE NO |
|---|---|---|
| 4.1 | Station-Level EV Charging Demand Forecasting system architecture | 7 |
| 5.1 | Cleaned session log reshaped into the hourly site-level table | 12 |
| 5.2 | Pipeline output — energy check, MAE and leakage test | 13 |
| 5.3 | Code architecture of the project | 13 |
| 5.4 | Phase status against the review timeline | 13 |
| 5.5 | Local web dashboard (localhost:8000) | 14 |
| 6.1 | Test-set MAE and RMSE by model, and Random Forest forecast versus actual demand | 16 |

# LIST OF TABLES


| TABLE NO | TITLE | PAGE NO |
|---|---|---|
| 2.1 | Summary of the literature reviewed | 4 |
| 3.1 | Weekly PBL progress log | 5 |
| 3.2 | Hardware and software requirements | 6 |
| 3.3 | Project configuration summary | 6 |
| 6.1 | Model evaluation results across iterations | 15 |
| 6.2 | Feature ablation of the final Random Forest | 17 |
| 6.3 | The five busiest stations, each with its own model | 17 |
| A.1 | Self and peer assessment | 23 |

# LIST OF ABBREVIATIONS

| ABBREVIATION | FULL FORM / DESCRIPTION |
|---|---|
| **ACN** | Adaptive Charging Network |
| **API** | Application Programming Interface |
| **CSV** | Comma-Separated Values |
| **EV** | Electric Vehicle |
| **JSON** | JavaScript Object Notation |
| **kWh** | Kilowatt-hour (unit of energy) |
| **kWh/h** | Kilowatt-hour per hour (demand rate) |
| **MAE** | Mean Absolute Error |
| **MAPE** | Mean Absolute Percentage Error |
| **ML** | Machine Learning |
| **PBL** | Project-Based Learning |
| **REST** | Representational State Transfer |
| **RF** | Random Forest |
| **RMSE** | Root Mean Squared Error |

# CHAPTER 1
# INTRODUCTION

## 1.1 Background

Electric vehicles are being adopted quickly, and the charging infrastructure that supports them is increasingly concentrated at workplaces, campuses and public hubs where many vehicles plug in during the same few hours of the day. For the operator of such a site, the practical question is not how much energy will be delivered over a month, but how much will be drawn in the next hour. That short-horizon number drives decisions that cannot wait: whether to stagger charging across connectors, how much grid capacity to reserve, and whether a demand charge is about to be triggered.

The difficulty is that charging demand at a single station is extremely sparse. A connector that serves a handful of vehicles a day sits idle for most hours, so a per-station series is dominated by zeros and carries almost no learnable signal. Aggregated to the level of a whole site, however, the same data show a clear and repeating structure: a weekday morning arrival peak, a decline through the afternoon, and near silence at night and over weekends. That structure is what makes the problem tractable.

Open session-level data make it possible to study this without access to a commercial operator's records. The Caltech ACN-Data dataset publishes individual charging sessions from real workplace sites through a public REST API, recording the station, the connection and disconnection times, and the energy delivered. This project uses that data to ask whether a machine learning model built only on backward-looking features can forecast the next hour of site-level demand more accurately than the obvious rule of thumb — repeating what happened at the same hour last week — and to do so under an evaluation that never lets the model see the future.

## 1.2 Driving Question

Can a machine learning model forecast next-hour, site-level EV charging demand from the Caltech ACN-Data dataset more accurately than the naive same-hour-last-week rule, using only backward-looking features and a strictly chronological evaluation?

The specific task is to reshape 6,490 charging sessions from 53 stations into an hourly site-level series, engineer lag features, train Linear Regression and Random Forest regressors, and

compare their Mean Absolute Error (MAE) and Root Mean Squared Error (RMSE) in kWh/h against the baseline, with the predictions and metrics served through a local dashboard.

## 1.3 Objectives

- To ingest real EV charging sessions from the Caltech ACN-Data REST API with a download script that retries after timeouts and saves every page as it arrives, and clean them into a tidy per-session table.
- To aggregate sessions into station-wise and whole-site hourly demand (kWh/h), because an average station has no demand in about 79% of hours and is not predictable on its own.
- To engineer backward-only features — hour, weekday, and demand 1 h, 2 h, 24 h and 168 h earlier — so that no future information leaks into any prediction, and to verify this with an automated test.
- To split the data chronologically (never shuffled), tune a Random Forest by time-series cross-validation, and compare it and Linear Regression against a same-hour-last-week baseline using MAE and RMSE.
- To serve the predictions and metrics through a small local web dashboard (/api/data → JSON).
- To document the PBL development process, limitations, and learning outcomes associated with the project.

## 1.4 Scope and Limitations

The project covers next-hour, site-level demand forecasting for the Caltech workplace charging site using 6,490 sessions (April–August 2018), six backward-looking features, two regression models, a naive baseline, MAE and RMSE evaluation, and a local dashboard. The five busiest stations are also modelled individually for comparison, but the project does not forecast every station and does not use weather, tariff or occupancy data. The results apply to the roughly three and a half months of data analysed and should not be generalised to other sites or seasons. Hourly demand is reconstructed from session records rather than metered, the model under-predicts sharp demand peaks, and the API connection can time out on long downloads, so the extract is limited to about 6,500 sessions and the downloader retries automatically.

# CHAPTER 2
# LITERATURE SURVEY

## 2.1 Related Approaches

### 2.1.1 Open EV charging datasets

ACN-Data (Lee, Li and Low, e-Energy ’19) releases session-level records — station, connection and disconnection times, and delivered energy — from workplace sites such as Caltech, together with a public REST API [1], [2]. It is the only data source used in this project.

### 2.1.2 Baseline and classical forecasting

A seasonal-naive forecast repeats the value from the same point in the previous cycle; for hourly demand with a weekly rhythm this is the value 168 hours earlier, and it is used here as the benchmark that the learned models must beat [3]. Linear Regression on lagged values is the simplest learned alternative [4].

### 2.1.3 Tree-based machine learning

Random Forest [5] averages many decision trees and handles nonlinear tabular relationships without feature scaling; it is the best-performing model in this project.

### 2.1.4 Evaluation

MAE is preferred to MAPE when the series contains zero or near-zero values [6]; RMSE, which penalises large errors more heavily, is reported alongside it. The scikit-learn library [7], [8], [9] provides the models and metrics, pandas [10] the data handling, and matplotlib [11] the figures.

## 2.2 Summary Table

| Ref. | Approach / Model | Dataset | Reported Result |
|---|---|---|---|
| [1] | ACN-Data open EV charging dataset | Real workplace charging sessions | Source of the 6,490 sessions used here. |
| [2] | ACN-Data REST API | Caltech site sessions | Token-authenticated, 100 rows per page; can time out on long downloads. |
| [3] | Seasonal-naive forecast (t − 168 h) | Hourly site demand | Baseline for weekly cycles; MAE 8.86 kWh/h. |
| [4] | Linear Regression on lag features | Hourly site demand | MAE 4.92 kWh/h (44.5% better). |
| [5] | Random Forest regressor | Hourly site demand | MAE 4.41 kWh/h (50.2% better). |

| Ref. | Approach / Model | Dataset | Reported Result |
|---|---|---|---|
| [6] | MAE versus MAPE | Series with zero or near-zero values | MAPE is undefined at zero and unstable near it; MAE and RMSE used. |
| [7] | scikit-learn | Tabular machine learning | Provides the models and the MAE and RMSE metrics. |

*Table 2.1 Summary of the literature reviewed*

# CHAPTER 3
# PROJECT PLANNING AND TEAM ORGANISATION

## 3.1 Weekly PBL Progress Log

| Week | Milestone / Task | Work Done | Status / Remarks |
|---|---|---|---|
| 1–2 | Problem framing and data acquisition | Framed the next-hour demand forecasting problem, set up the repository, and fetched sessions from the ACN-Data REST API with a downloader that retries after timeouts and saves each page to CSV as it arrives. | Completed. |
| 3 | Sessions to hourly series | Cleaned sessions, spread each session's kWh over its charging window, summed kWh per hour across stations with zero-filled quiet hours, and checked that total energy is conserved. | Completed. |
| 4–5 | Baseline, MAE harness and Linear Regression | Built the same-hour-last-week baseline and MAE evaluation first, then trained Linear Regression on lag features. | Completed; 44.5% better than baseline (MAE 4.92). |
| 6–7 | Random Forest, features and web prototype | Engineered hour, weekday and lag features (1 h, 2 h, 24 h, 168 h), trained Random Forest, and built the local dashboard. | Completed; 51.0% better than baseline with fixed parameters. |
| 8–9 | Error analysis, testing, report and packaging | Tuned the Random Forest by time-series cross-validation, added RMSE, error analysis by hour, weekday and station, a feature ablation, the leakage test, README and requirements; report updated to the final run. | Completed; tuned forest 50.2% better than baseline. |

*Table 3.1 Weekly PBL progress log*

## 3.2 Requirements

Dataset source: Caltech ACN-Data (https://ev.caltech.edu/acndata), fetched through its public token-authenticated REST API for the caltech site; a free API token is obtained by registering on the ACN-Data website, and the public demonstration token was used. The extract contains 6,500 downloaded sessions, of which 6,490 remain after cleaning, from 53 stations (25 April to 9 August 2018), reshaped into 2,555 hourly rows of whole-site demand (2,387 after adding lag features).

| Category | Requirement |
|---|---|
| Processor / RAM | A standard laptop (Windows 11 used); no GPU required. |

| Category | Requirement |
|---|---|
| Programming language | Python 3.x |
| Libraries / frameworks | pandas, NumPy, scikit-learn, requests, joblib, matplotlib (plots), Chart.js (dashboard charts) |
| Development environment | VS Code on Windows 11; plain Python scripts (no notebooks); GitHub repository |

*Table 3.2 Hardware and software requirements*

| Category | Configuration |
|---|---|
| Dataset | Caltech ACN-Data; 6,490 cleaned sessions (6,500 downloaded), 53 stations, April–August 2018; 2,555 hourly rows (2,387 with lag features) |
| Model | Baseline (t − 168 h), LinearRegression, RandomForestRegressor(n_estimators=200, min_samples_leaf=1, max_depth=None, random_state=42), chosen by TimeSeriesSplit cross-validation |
| Preprocessing | Each session's energy spread over its charging window, hourly aggregation with zero-fill; backward-only lag features; chronological 80:20 split |
| Interface | Local web dashboard (Python http.server + Chart.js) at http://localhost:8000; /api/data returns JSON |

*Table 3.3 Project configuration summary*

## 3.3 Feasibility

The project is feasible within the PBL timeframe (a nine-phase plan spread over about nine weeks) and the resources available to the team. Technically, the pipeline relies on mature open-source libraries (pandas, scikit-learn, requests, matplotlib) and a free public dataset, so no specialised infrastructure or licensing cost is required; it runs on a standard Windows 11 laptop with no GPU, and the whole pipeline — download, reshaping, tuning and training on about 2,387 hourly rows — completes in minutes. Operationally, the work divides cleanly into data acquisition, reshaping, feature engineering, modelling and dashboard stages, each implemented as its own script, which map onto the two-member team's weekly milestones in Table 3.1; all nine phases are complete. The main risks were the API connection timing out on long downloads, the sparsity of individual stations, and the short history of under four months; these were mitigated by a downloader that retries and resumes; the main mitigation was forecasting the whole site and using a chronological split.

# CHAPTER 4
# ITERATIVE DESIGN AND DEVELOPMENT

## 4.1 System Architecture

The pipeline is organised as five one-way layers (Figure 4.1). The external source is the Caltech ACN-Data REST API (token authentication, 100 sessions per page). The ingestion layer downloads the sessions page by page, retrying each page up to five times, and appends every page to a CSV file immediately so that a timeout cannot lose data already received; a second script cleans the file into a tidy sessions table (station, connection, disconnection and charge-end time, kWh) in Pacific time. The transformation layer spreads each session's energy over the hours of its charging window, sums it per hour for every station and for the whole site, and zero-fills hours with no charging, giving 2,555 hourly rows; a validation check confirms that total energy is unchanged by the reshaping (58,785.12 kWh in and out). The modelling layer builds backward-only features and trains the baseline, Linear Regression and a cross-validated Random Forest. The presentation layer reports MAE and RMSE against the baseline, runs the leakage test, and serves the results as JSON from /api/data to a browser dashboard.

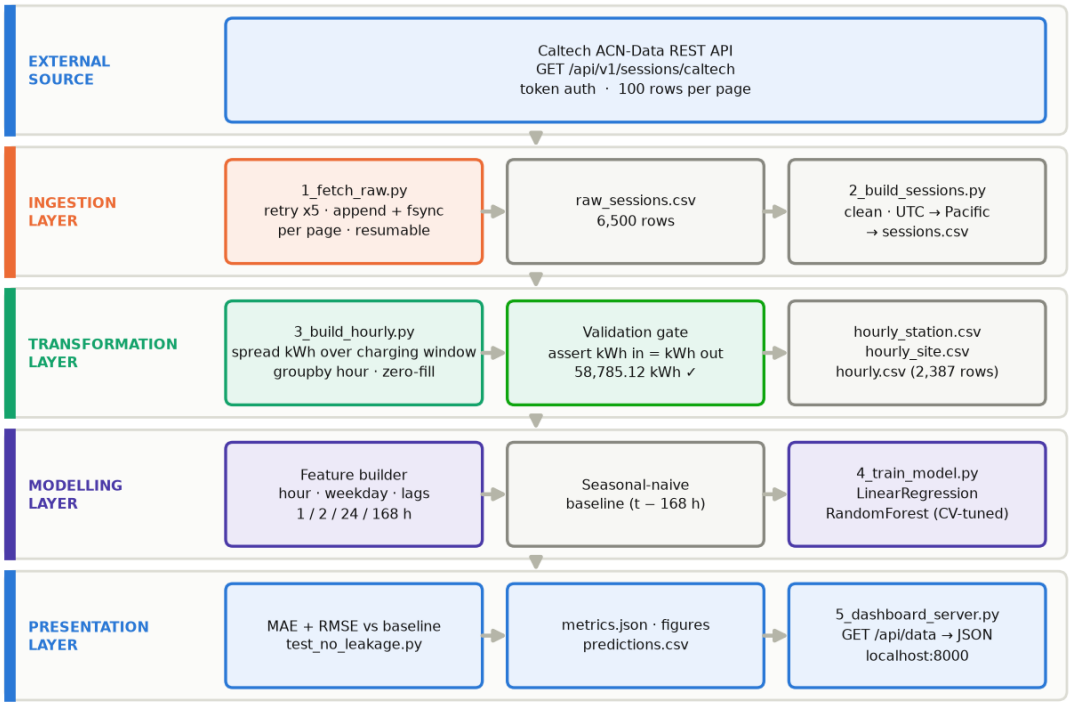



*Figure 4.1 Station-Level EV Charging Demand Forecasting system architecture*

## 4.2 Iteration 1 — Baseline

The simplest version fetches the sessions, reshapes them into an hourly site-level series, and evaluates the naive rule "demand at hour t equals demand at hour t − 168" (same hour last

week) on the most recent 20% of hours. This baseline and the evaluation harness were built before any machine learning model so that every later model has a fixed bar to beat. On the 478 held-out hours the baseline gives MAE = 8.86 kWh/h and RMSE = 12.67 kWh/h.

Linear Regression was then added as the first learned model on six backward-only features, giving MAE = 4.92 kWh/h (RMSE 6.63), which is 44.5% better than the baseline.

## 4.3 Iteration 2 — Refinement

The Linear Regression model was complemented by a Random Forest regressor with fixed hyperparameters (300 trees, min_samples_leaf = 2) on the same six backward-only features (hour, weekday, lag_1h, lag_2h, lag_24h, lag_168h). The Random Forest can model nonlinear interactions — for example, a morning arrival peak that depends on both the hour of day and the demand an hour earlier — and reduced MAE to 4.34 kWh/h, a 51.0% improvement over the baseline. A local web dashboard was then added to serve actual and predicted demand and the summary metrics as JSON from /api/data.

**Iteration 3 — cross-validated tuning.** A small grid (n_estimators 200 or 300, min_samples_leaf 1, 2 or 4, max_depth unlimited or 12) was searched with four-fold TimeSeriesSplit cross-validation on the training period only, so the test hours played no part in the choice. The search selected 200 trees, min_samples_leaf = 1 and unlimited depth (cross-validated MAE 4.42 kWh/h). On the held-out hours this tuned forest scored MAE 4.41 kWh/h, essentially the same as the fixed-parameter forest (4.34), so tuning did not improve the test score. It is kept as the final model because the selection rule was fixed in advance and does not look at the test period.

## 4.4 Final Approach

The final approach is a Random Forest regressor that predicts site-level kWh in hour t from six clues that all look backward: hour of day, weekday, and demand 1 h, 2 h, 24 h and 168 h earlier. Random Forest builds many decision trees and averages their predictions; the final model uses n_estimators=200, min_samples_leaf=1, max_depth=None and random_state=42. In the final run the most useful clues were demand one hour earlier (importance 0.790), demand one week earlier (0.079), hour of day (0.070) and demand one day earlier (0.028), which reflects that a car that began charging in the last hour is usually still charging, on top of a regular daily and weekly rhythm.

## 4.5 Training Procedure

1. Read the hourly feature table (hourly.csv) produced by the pipeline.
2. Select the six feature columns (hour, weekday, lag_1h, lag_2h, lag_24h, lag_168h) and the target demand_kwh.
3. Sort the rows by time and split chronologically: the earliest 80% for training (1,909 rows, 2 May 2018 to 20 July 2018) and the latest 20% for testing (478 rows, 20 July 2018 to 9 August 2018); the data are never shuffled.
4. Compute the baseline prediction for each test hour as the value of lag_168h (same hour last week).
5. Fit LinearRegression on the training rows.
6. Fit RandomForestRegressor twice on the training rows: once with fixed settings (300 trees, min_samples_leaf=2, random_state=42), and once with settings chosen by GridSearchCV over TimeSeriesSplit (4 folds), scored by MAE on the training period only.
7. Predict the test rows and score every model with MAE and RMSE in kWh/h (MAPE is not used because demand is zero or close to zero in many hours).
8. Save the final (tuned) Random Forest (joblib), the metrics (metrics.json) and the test predictions (predictions.csv) for the dashboard, together with the error by hour and weekday, the feature ablation, and the per-station results for the five busiest stations.

**Training details.** A single chronological train/test split is used; model selection uses four-fold TimeSeriesSplit cross-validation inside the training period, so there is no separate validation set. Hyperparameters were tuned over a 12-combination grid; no feature scaling is applied; and no loss function, optimizer or epochs are involved because Linear Regression uses least squares and Random Forest does not use epoch-based gradient training.

# CHAPTER 5
# IMPLEMENTATION

## 5.1 Module Description

**Data Ingestion Module (1_fetch_raw.py, 2_build_sessions.py):** pages through the ACN-Data REST API with the API token (100 sessions per request) until about 6,500 sessions are saved. Each page is retried up to five times, waiting between one and five minutes after a failure, and is appended to raw_sessions.csv and flushed to disk before the next request; progress is recorded in download_state.json so an interrupted run resumes where it stopped. The second script cleans the raw file into a tidy per-session table (station_id, connection_time, disconnect_time, charge_end_time, kwh_delivered), converts times to Pacific time, and drops duplicates, malformed records, non-positive energy, sessions that end before they start, and sessions longer than 72 hours (10 of 6,500).

**Hourly Aggregation and Feature Module (3_build_hourly.py):** spreads each session's kWh evenly across the hours of its charging window (connection to charge-end time), sums it per hour for every station and for the whole site, and zero-fills hours with no charging. It asserts that total energy is unchanged, writes the station-wise and site-wide hourly tables and the dataset statistics, then adds hour, weekday and lag features (1 h, 2 h, 24 h, 168 h) through one shared function and drops the first week that lacks full lag history.

**Model Training Module (4_train_model.py):** splits the hourly table chronologically (80:20), evaluates the same-hour-last-week baseline, trains Linear Regression and a fixed-parameter Random Forest, tunes a second Random Forest by time-series cross-validation, and scores each with MAE and RMSE. It also computes error by hour of day and weekday, a feature ablation, and per-station results for the five busiest stations, and saves the final model, metrics.json and predictions.csv.

**Dashboard Module (5_dashboard_server.py):** a Python http.server application on localhost:8000 that serves a plain HTML/JavaScript page (Chart.js, bundled so it works offline) at / and a JSON payload at /api/data combining predictions.csv, metrics.json, stats.json and station_results.csv. It listens on the local machine only.

**Test and Utility Scripts (test_no_leakage.py, generate_dataset.py, make_figures.py):** test_no_leakage.py checks that every lag feature equals demand at the earlier hour, that the training period ends before the test period begins, that the checker detects a deliberately leaky

feature, and that a forest trained on shuffled targets is clearly worse. generate_dataset.py creates a small synthetic raw_sessions.csv in the API's schema so the pipeline can be tried without a token (it is not used for any reported result), and make_figures.py redraws the report figures from the pipeline's output.

## 5.2 Key Code Snippets

Only the four steps that carry the method are reproduced here — hourly aggregation with zero-fill, backward-only feature construction, the chronological split with the naive baseline, and model training and scoring. The complete scripts are listed in the Appendix.

### (a) Reshaping sessions into zero-filled hourly demand (3_build_hourly.py)

```
# spread each session's energy over the hours of its charging window
span = pd.date_range(start.floor('h'), end.ceil('h'), freq='h')[:-1]
share = energy / len(span)
# ... one (station, hour, share) row per hour; then sum and zero-fill
wide = (spread.groupby(['hour', 'station_id'])['demand_kwh'].sum()
              .unstack('station_id')
              .reindex(full_range, fill_value=0.0).fillna(0.0))
site = wide.sum(axis=1)
# validation gate: reshaping must neither lose nor create energy
assert abs(energy_in - energy_out) < 1e-6
```

### (b) Backward-only feature engineering (common.py)

```
out['hour_of_day'] = out['hour'].dt.hour
out['weekday']     = out['hour'].dt.weekday
# every lag looks strictly backward -- no future information can leak
for lag in (1, 2, 24, 168):
    out[f'lag_{lag}h'] = out['demand_kwh'].shift(lag)
# drop the first week, which has no complete 168-hour history
out = out.dropna()
```

### (c) Chronological split and the same-hour-last-week baseline (4_train_model.py)

```
FEATURES = ['hour_of_day', 'weekday', 'lag_1h', 'lag_2h',
            'lag_24h', 'lag_168h']
df = df.sort_values('hour')          # time order, never shuffled
split = int(len(df) * 0.8)
train, test = df.iloc[:split], df.iloc[split:]
# baseline: repeat the value from the same hour last week
baseline_mae = mean_absolute_error(test['demand_kwh'], test['lag_168h'])
```

**(d) Tuning by cross-validation, training and scoring the models (4_train_model.py)**

```
lin = LinearRegression().fit(X_train, y_train)
search = GridSearchCV(RandomForestRegressor(random_state=42),
    {'n_estimators': [200, 300], 'min_samples_leaf': [1, 2, 4],
     'max_depth': [None, 12]},
    cv=TimeSeriesSplit(n_splits=4),
    scoring='neg_mean_absolute_error').fit(X_train, y_train)
rf = RandomForestRegressor(random_state=42, **search.best_params_)
rf.fit(X_train, y_train)      # parameters chosen on the training period only
mae  = mean_absolute_error(y_test, rf.predict(X_test))
rmse = np.sqrt(mean_squared_error(y_test, rf.predict(X_test)))
joblib.dump(rf, 'models/random_forest.joblib')
```

## 5.3 User Interface / Demo

The prototype is a local web page (http://localhost:8000) started with 5_dashboard_server.py. It shows the MAE and RMSE of the baseline, Linear Regression and both Random Forests, a line chart of actual versus predicted demand on the test hours with a week-window selector, the error by hour of day, the feature importances, the feature ablation, and a table for the five busiest stations; the same data are available as JSON at /api/data (Figure 5.5). The other figures show the data reshaping, the pipeline run, the code layout and the phase status against the review timeline.

**data/sessions.csv — cleaned transaction log**

6,490 sessions · 53 stations · one row per car

| station | connect_time | charge_end | kwh |
|---|---|---|---|
| 2-39-78-362 | 2018-04-25 04:08:04 | 06:20:10 | 7.932 |
| 2-39-95-27 | 2018-04-25 06:45:10 | 09:44:15 | 10.013 |
| 2-39-79-380 | 2018-04-25 06:45:50 | 07:51:44 | 5.257 |
| 2-39-79-379 | 2018-04-25 07:37:06 | 09:05:22 | 5.177 |
| 2-39-79-381 | 2018-04-25 07:40:34 | 10:40:30 | 10.119 |
| 2-39-139-28 | 2018-04-25 07:43:50 | 09:18:28 | 7.91 |

3_build_hourly.py ↓ spread kWh over charging window · groupby(hour) · sum · reindex full range

**data/hourly_site.csv — reshaped hourly series**

2,555 hours · quiet hours are real zeros, not missing

| hour | demand_kwh |
|---|---|
| 2018-04-25 04:00:00 | 2.644 |
| 2018-04-25 05:00:00 | 2.644 |
| 2018-04-25 06:00:00 | 7.776 |
| 2018-04-25 07:00:00 | 17.400 |
| 2018-04-25 08:00:00 | 35.418 |
| 2018-04-25 09:00:00 | 53.191 |

*Figure 5.1 Cleaned session log (6,490 sessions, 53 stations) reshaped into the hourly site-level table (2,555 hours)*

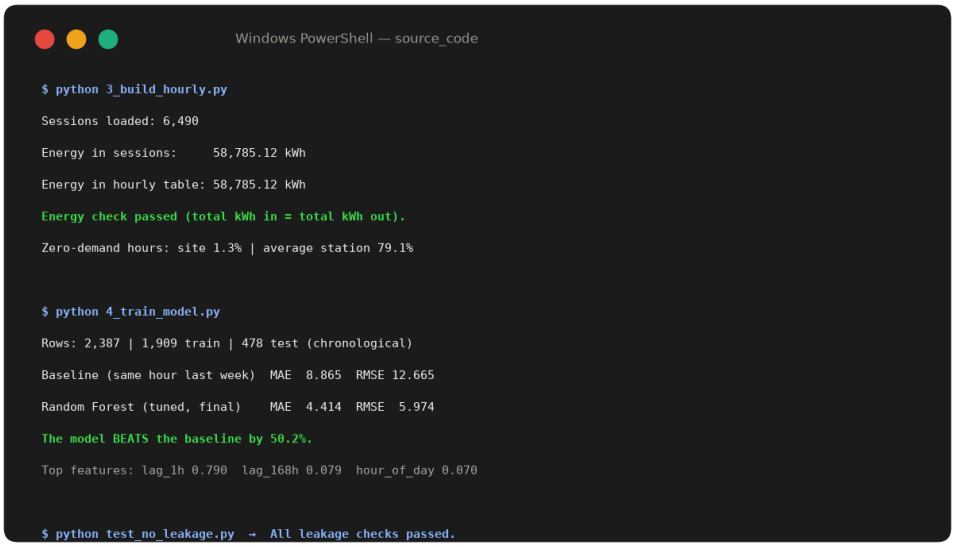



*Figure 5.2 Pipeline output — energy check passed, Random Forest MAE 4.41 vs baseline 8.86 kWh/h, leakage test passed*




*Figure 5.3 Code architecture of the project (download, reshape, forecasting scripts, data and web folders)*

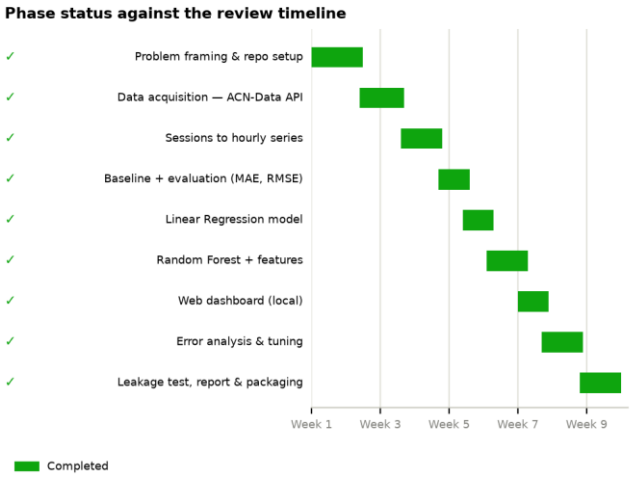



*Figure 5.4 Phase status against the review timeline (all nine phases completed)*




*Figure 5.5 Local dashboard at http://localhost:8000: accuracy cards, forecast versus actual, error by hour of day and feature importance*

# CHAPTER 6
# RESULTS AND DISCUSSION

## 6.1 Evaluation Metrics

MAE (Mean Absolute Error) is the average absolute difference between the actual and the predicted hourly demand, in kWh/h; lower is better and it is expressed in the same units as demand. RMSE (Root Mean Squared Error) is the square root of the average squared difference; it is also in kWh/h but penalises large misses more heavily, so a wide gap between RMSE and MAE signals occasional large errors. MAE is the headline metric and MAPE is not used: demand is zero or close to zero in many hours (the site series is exactly zero in 1.3% of hours and below 1 kWh/h in 3%), where percentage error is undefined or explodes. Improvement over the baseline is (baseline MAE − model MAE) / baseline MAE × 100. All metrics are computed on the held-out, chronologically latest 20% of hours (478 rows).

## 6.2 Results Across Iterations

| Model | MAE (kWh/h) | RMSE (kWh/h) | Improvement vs. baseline (MAE) |
|---|---|---|---|
| Baseline (same hour last week) — reference rule | 8.86 | 12.67 | — |
| Linear Regression — learned on lag features | 4.92 | 6.63 | 44.5% |
| Random Forest (fixed parameters) | 4.34 | 5.91 | 51.0% |
| Random Forest (cross-validated, final) | 4.41 | 5.97 | 50.2% |

*Table 6.1 Model evaluation results across iterations*

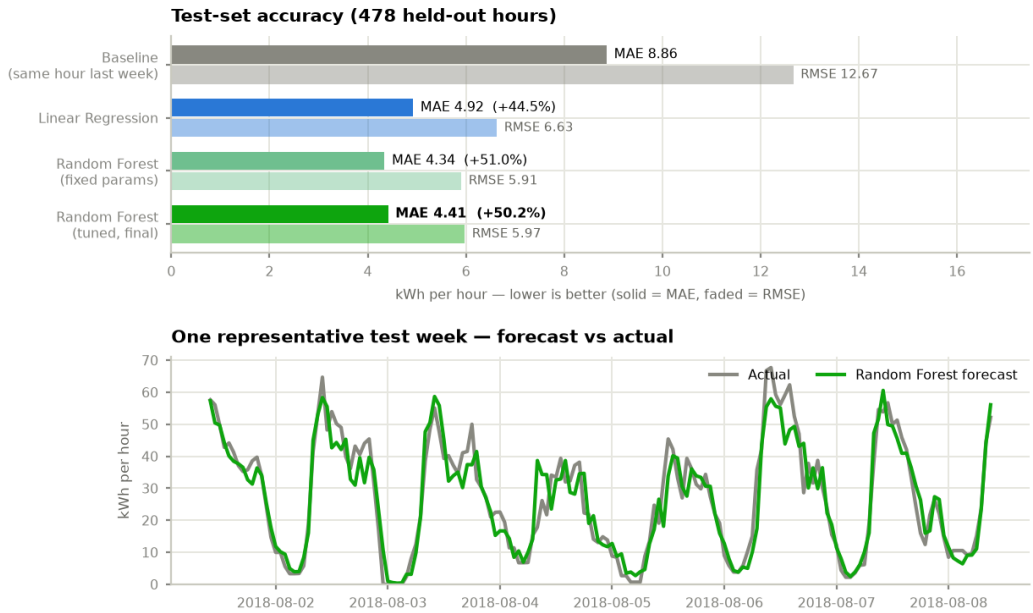



*Figure 6.1 Test-set MAE and RMSE of the baseline, Linear Regression and both Random Forests (top) and Random Forest forecast versus actual demand for one representative test week, 1–8 August 2018 (bottom)*

## 6.3 Discussion

The evaluation respects time: the data are split chronologically and every feature looks backward, which test_no_leakage.py verifies, so the models are tested strictly on the future. Linear Regression improves on the baseline by 44.5% and the Random Forest by about 50.2% (MAE 4.41 versus 8.86 kWh/h; RMSE 5.97 versus 12.67 kWh/h), so recent demand carries far more information than repeating last week's value. Feature importance ranks demand one hour earlier first (0.790), followed by demand one week earlier (0.079), hour of day (0.070) and demand one day earlier (0.028). Errors are largest around 09:00, 10:00, 11:00 and 18:00 (MAE 6.0–6.7 kWh/h) and smallest around 02:00, 03:00, 04:00 and 05:00 (1.1–2.0 kWh/h). In Figure 6.1 the forecast follows the daily cycle closely but under-predicts the sharpest peaks: over the busiest 10% of test hours it is on average 5.5 kWh/h too low.

The improvement should be read with care, and the feature ablation in Table 6.2 shows why. Hourly demand is reconstructed from sessions by spreading each session's energy over its charging window, which uses the session's end time; a live meter would not know that in advance, so the 1 h and 2 h lags can carry somewhat more than they would in deployment. With those two lags removed the forest scores MAE 7.63 kWh/h, still 13.9% better than the baseline, and calendar features alone (hour and weekday) reach 7.40 kWh/h (16.5% better). The conservative reading is therefore that the forest beats last week's value by roughly 14–

17% without the recent lags, and the 50.2% figure is an upper bound that depends on the demand reconstruction. Tuning did not help either: the cross-validated forest (4.41) is marginally worse than the fixed-parameter one (4.34), a difference too small to mean anything on 478 hours, and no significance test was run.

| Features used | MAE (kWh/h) | RMSE (kWh/h) | Improvement vs. baseline (MAE) |
|---|---|---|---|
| All six features (final) | 4.41 | 5.97 | 50.2% |
| Without lag_1h / lag_2h | 7.63 | 10.88 | 13.9% |
| Calendar only (hour, weekday) | 7.40 | 10.87 | 16.5% |
| Daily + weekly lags only | 8.72 | 11.85 | 1.6% |

*Table 6.2 Feature ablation: the final Random Forest refitted on feature subsets (478 test hours)*

The pipeline also checks that total energy is conserved when sessions are reshaped into hourly demand (58,785.12 kWh in the sessions and in the hourly table), so the aggregation step neither loses nor creates energy.

Fitting the same settings to each of the five busiest stations (Table 6.3) gives an MAE of 0.48–0.63 kWh/h against 0.92–1.16 kWh/h for each station's own same-hour-last-week baseline (41–55% better), but these stations still have no demand in 55–71% of hours and the absolute numbers are small, so station-level forecasts are far less useful than the site total. Linear Regression is as good or better than the forest at 2 of the five stations (2-39-91-437 and 2-39-79-377), which is typical of sparse series where a simple model is hard to beat.

| Station | Zero-demand hours | MAE: baseline / Random Forest (kWh/h) | Improvement vs. baseline (MAE) |
|---|---|---|---|
| 2-39-139-28 | 54.6% | 1.16 / 0.63 | 45.7% |
| 2-39-91-437 | 69.3% | 1.06 / 0.63 | 40.7% |
| 2-39-131-30 | 58.7% | 0.97 / 0.49 | 49.3% |
| 2-39-95-27 | 70.9% | 1.07 / 0.48 | 54.8% |
| 2-39-79-377 | 67.9% | 0.92 / 0.49 | 46.6% |

*Table 6.3 The five busiest stations, each with its own model and its own same-hour-last-week baseline*

## 6.4 Limitations

The main limitations are the short history (about three and a half months, April–August 2018, from a single Caltech site), a test period of only 478 hours (about 20 days), and the sparsity of

individual stations (about 79% zero-demand hours on average), which is why the headline model forecasts site-level demand. Hourly demand is reconstructed by spreading each session's energy evenly over its charging window, which smooths real charging curves and uses information (the session's end time) that a live system would not have; this inflates the value of the 1 h and 2 h lags, as the ablation shows. The Random Forest under-predicts sharp demand peaks. Tuning was a small 12-combination grid and did not improve the held-out score, and no significance test or repeated cross-validation of the final comparison was run. The download is limited to about 6,500 sessions, and the ACN-Data API can time out on long requests, which the retry logic handles but which limits how much data is practical to fetch. Results may not transfer to other sites, seasons or years, and external factors such as weather, holidays, tariffs and fleet changes are not represented in the features.

# CHAPTER 7
# TEAM REFLECTION AND LEARNING OUTCOMES

## 7.1 Individual Reflections

**M PRAVEEN VIGNESH:** Worked on data acquisition from the ACN-Data API, the retry-safe downloader, reshaping sessions into the hourly series, and backward-only feature engineering. Learned how a raw transaction log becomes a forecasting-ready time series and why every feature must look only backward to avoid leakage. A key challenge was the API connection timing out on long downloads, which led to retrying each page and saving it to disk as it arrives.

**J FARHAN SOHAIL:** Worked on model training, MAE and RMSE evaluation against the baseline, the local dashboard, and documentation. Learned why a time-ordered split and MAE (rather than MAPE) are appropriate for demand data containing zero and near-zero hours. A key challenge was the model under-predicting sharp demand peaks.

Both members reviewed the results together and prepared the review slides, the poster and this report, and learned the importance of a fixed baseline and a held-out future test period when judging a forecasting model.

## 7.2 Team Learning

The project divides the work into data acquisition, reshaping, modelling, dashboard and documentation, with one script per pipeline stage, which made it easy to share work and test each stage on its own. A useful team practice was building the baseline and MAE harness first, so every later change could be judged against a fixed bar, and keeping all features strictly backward-looking, which the leakage test now checks automatically. If the PBL cycle were restarted, more time would be reserved for gradient boosting, a longer history from more sites, and a demand series built from metered power rather than reconstructed from sessions, because the feature ablation showed how much the headline result depends on that reconstruction. Mentor feedback should be recorded against the actual review meetings; no specific mentor comments were supplied, so none are attributed here.

## 7.3 Course Outcomes — Evidence Summary

**Data handling:** the project demonstrates fetching data from a REST API with timeout-safe retries and incremental saving, cleaning session records, reshaping them into station-wise and site-wide hourly time series, and validating the reshaping with an energy-conservation check.

**Machine learning:** it applies lag-feature engineering, chronological train-test splitting, time-series cross-validation for hyperparameter tuning, Linear Regression and Random Forest regression, and comparison with a naive baseline.

**Evaluation:** it uses MAE and RMSE on held-out future hours, justifies MAE over MAPE for data with zero and near-zero hours, breaks the error down by hour, weekday and station, and uses a feature ablation to show where the accuracy comes from.

**Software integration:** it connects the trained model's predictions to a local web dashboard through a JSON API and covers the feature pipeline with an automated leakage test.

**Teamwork:** the one-script-per-stage structure supports separate responsibilities, as recorded in the weekly log in Table 3.1.

**Ethical awareness:** the report treats the output as a decision-support prototype, states its limitations, and notes the need for broader validation before operational use.

# CHAPTER 8
# CONCLUSION AND FUTURE SCOPE

## 8.1 Conclusion

The project built a complete pipeline that forecasts next-hour, site-level EV charging demand for the Caltech workplace charging site from 6,490 real sessions across 53 stations. A retry-safe script downloads the sessions, which are reshaped into 2,555 hourly rows; six backward-only features are engineered and the data are split chronologically. On 478 held-out hours the cross-validated Random Forest achieved an MAE of 4.41 kWh/h and an RMSE of 5.97 kWh/h, 50.2% better in MAE than the same-hour-last-week baseline (8.86 kWh/h), and Linear Regression achieved 4.92 kWh/h (44.5% better). The predictions are served through a local dashboard at http://localhost:8000. The work answers the driving question for this dataset: learned models using lag features beat the naive rule, mainly through recent-hour demand. Without the 1 h and 2 h lags the Random Forest's gain falls to 13.9% (MAE 7.63 kWh/h), and the model still under-predicts sharp peaks.

## 8.2 Future Scope

- Try Gradient Boosting and a wider tuning search, and test whether the differences between models are statistically meaningful with repeated time-series cross-validation.
- Build the demand series from metered power (the ACN-Data time-series endpoint) instead of reconstructing it from session records, to remove the look-ahead in the short lags.
- Forecast further ahead than one hour, where the short lags cannot be used, and extend the per-station models beyond the five busiest stations.
- Use a longer history and additional charging sites, and add external features such as weather and holidays.
- Deploy the dashboard as a hosted web application rather than a local server.

# REFERENCES


[1] Z. J. Lee, T. Li, and S. H. Low, "ACN-Data: Analysis and applications of an open EV charging dataset," in Proc. ACM e-Energy '19, 2019.

[2] Caltech, "ACN-Data public API." [Online]. Available: https://ev.caltech.edu/acndata

[3] R. J. Hyndman and G. Athanasopoulos, Forecasting: Principles and Practice, 3rd ed. Melbourne, Australia: OTexts, 2021.

[4] T. Hastie, R. Tibshirani, and J. Friedman, The Elements of Statistical Learning, 2nd ed. New York, NY, USA: Springer, 2009.

[5] L. Breiman, "Random forests," Machine Learning, vol. 45, no. 1, pp. 5–32, 2001.

[6] R. J. Hyndman and A. B. Koehler, "Another look at measures of forecast accuracy," International Journal of Forecasting, vol. 22, no. 4, pp. 679–688, 2006.

[7] F. Pedregosa et al., "Scikit-learn: Machine learning in Python," Journal of Machine Learning Research, vol. 12, pp. 2825–2830, 2011.

[8] scikit-learn developers, "RandomForestRegressor," scikit-learn documentation. [Online]. Available: https://scikit-learn.org/

[9] scikit-learn developers, "LinearRegression," scikit-learn documentation. [Online]. Available: https://scikit-learn.org/

[10] pandas development team, "pandas documentation." [Online]. Available: https://pandas.pydata.org/docs/

[11] J. D. Hunter, "Matplotlib: A 2D graphics environment," Computing in Science & Engineering, vol. 9, no. 3, pp. 90–95, 2007.

# APPENDIX

**A.1 Full source code:** common.py, 1_fetch_raw.py, 2_build_sessions.py, 3_build_hourly.py, 4_train_model.py, 5_dashboard_server.py, test_no_leakage.py, make_figures.py, generate_dataset.py, the web/ folder, README.md and requirements.txt. GitHub: https://github.com/Praveen-Vignesh/EV-Charging-station-demand-prediction-

**A.2 Complete weekly PBL log:** the week-by-week record is given in Table 3.1. Physical review sheets signed by the mentor are retained with the team and attached with the hard copy of this report.

### A.3 Self and Peer Assessment

*Table A.1 Self and peer assessment*

| Team Member | Self-Rated Contribution (%) | Peer-Rated Contribution (%) | Remarks |
|---|---|---|---|
| M PRAVEEN VIGNESH | 50 | 50 | Data acquisition, hourly reshaping and feature engineering. |
| J FARHAN SOHAIL | 50 | 50 | Model training, evaluation, dashboard and documentation. |